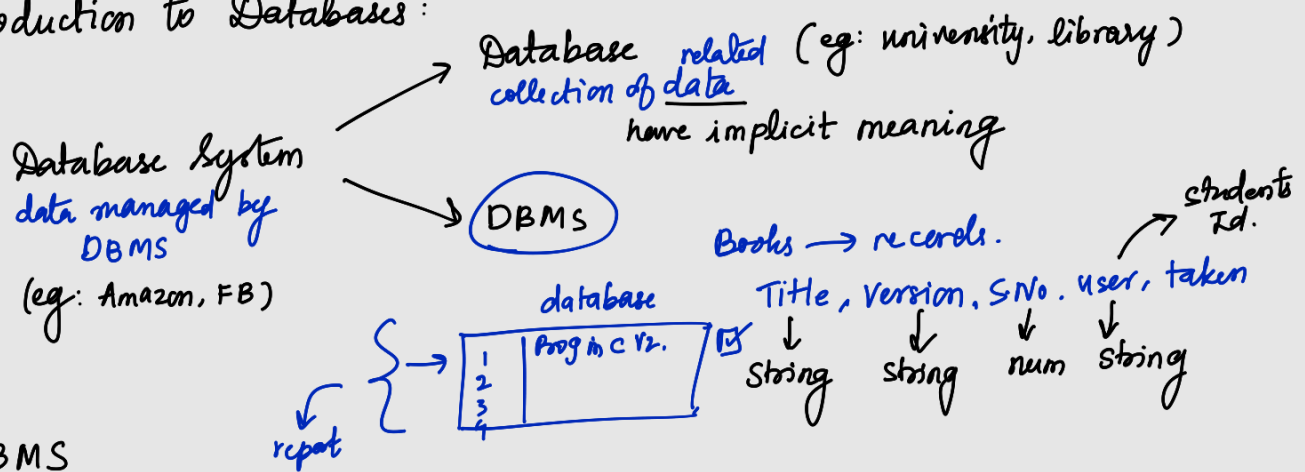


DBMS MODULE 1

• Introduction to Databases:



- DBMS

Collection of programs that enables users to create and maintain a database. It is a general purpose software system that facilitates the processes of defining, constructing, manipulating and sharing databases among various users and applications.

1. Define: specifying datatype, structures, constraints
2. Construct: Storing the data in some storage medium controlled by DBMS
3. Manipulate: Query the database, updating, generating reports from the data
4. Protection and Maintenance of DB: against hardware, software malfunction, unauthorized, malicious access.
5. Sharing: Allows multiple users and programs to access the database

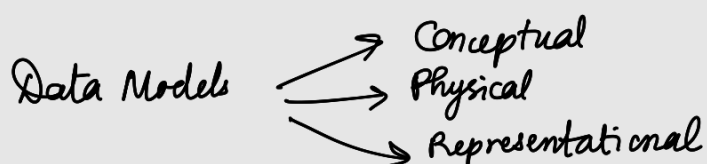
Metadata: Database definition or descriptive information stored by DBMS in the form of database catalog or dictionary.

Title → String [60]
S.No → Number [6]

- Database System Concepts:

1. Data Models:

Collection of concepts that can be used to describe how the logical structure of a database is modelled. They introduce abstraction in DBMS. It defines data types, relationships, constraints that we apply to the data or how the data is connected with each other, processed and stored inside systems.



a) Conceptual data models

- High level
- provides concepts close to many users perceive data
- uses entity, attributes and relationships

b) Physical data models → storage

- low level
- not for end users
- provide concepts that describe how data is stored in computer storage media.

c) Representational data model

- provide concepts that can be easily understood by end users but that are not too far removed from the way data is organized in computer storage.

- relational data model
- record based data models.

2. Schema → blueprint

Description of database

STUDENT

R_no	Name	Class	Marks
------	------	-------	-------

COURSE

C_id	Course-name	Credit	Dept
------	-------------	--------	------

database

R-no	Name	class	Marks
1			
2			
3			

Schema diagram

3. Instances

Data in the database at a particular moment in time is called a database state / snapshot that also refers to the current set of instances in database.

In a given database state, each schema construct has its own set of instances.

STUDENT table

eg:

R_no	Name	class	Marks
001	Ammu	A	29
002	Anita	A	35

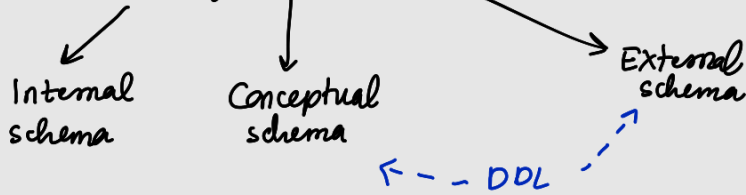
instance T1

C_id	Course-name	Credit	Dept
A01	COA	3	CSE
B01	DBMS	3	CSE

- Three Schema Architecture

Main purpose is to provide users with an abstract view of data and separate user application from physical database.
 hide complexity

Three Levels of Abstraction



1. Internal schema / Physical schema:

- Describes physical storage structure and details (access paths, ways to access it)

- Uses physical data model
- no interaction from user
- actual data stored.

2. Conceptual schema:

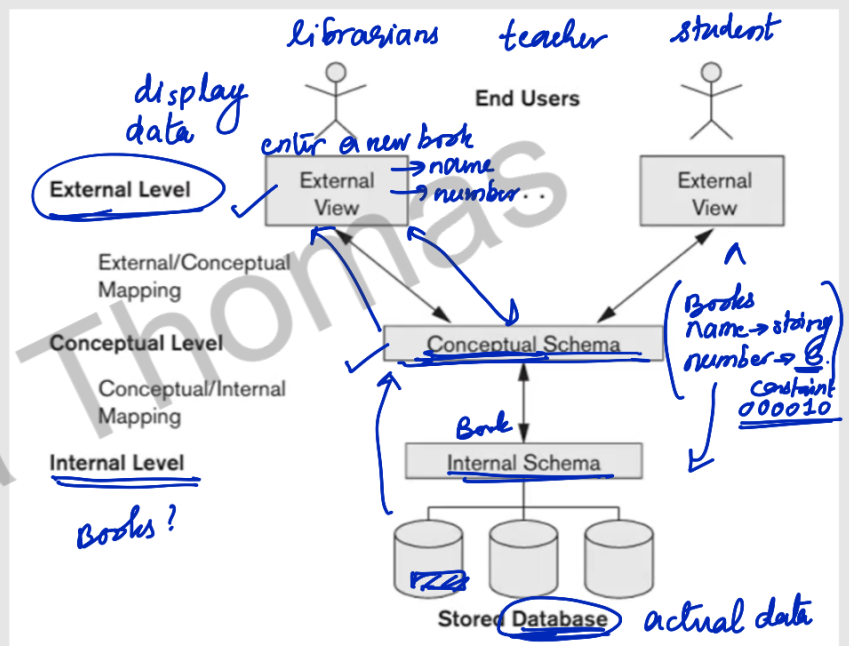
- describes the structure of the entire database

- hides all physical data storage detail and focuses on relns, data types, operations, constraints

3. External schema:

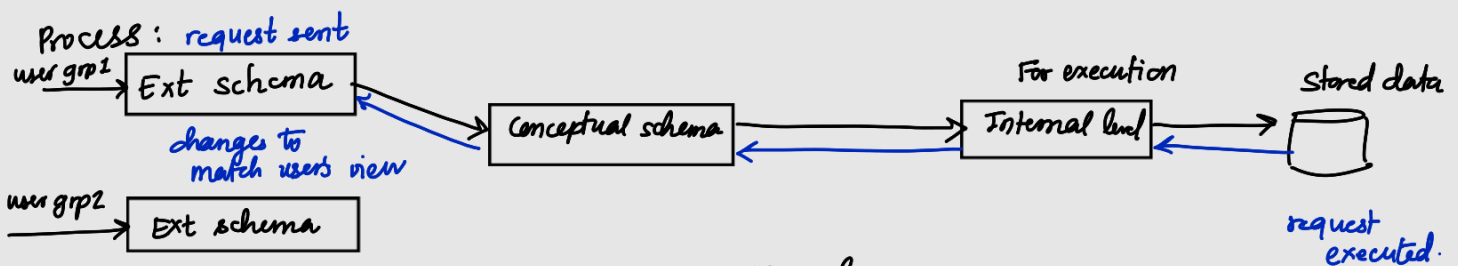
- Closest interface to the user
- Each external schema

describes part of the database that a part. user group is interested and hides the rest from them.



Books		
S.No	name	Topic

Any given database has 1 conceptual and 1 physical schema but may have several external schemas.



* mappings b/w schemas.

- Data Independence

Ability to modify a schema definition in one level without affecting the schema definition in next higher level.

1. Physical data independence

Ability to modify a schema followed at physical level without affecting the schema definition at logical level.

eg: creating additional access structures

Exists in environments where physical details like exact location of data, hardware details like storage encoding, placement etc. are hidden from user.

2. Logical data independence

Ability to modify a schema followed at logical level without affecting the schema definition at external level.

- Database Languages

1. Data Definition Language (DDL)

- used to define database structure or schema

- includes commands like CREATE, ALTER, DROP, TRUNCATE

2. Data Manipulation Language (DML) → record level / data

- used to insert, delete and update data

- commands like SELECT, INSERT, UPDATE, DELETE

Low Level DML

- retrieves one record at a time

- programmer must specify how the data is retrieved

- embedded inside a prog. language (loops)
C, Java

High level DML

- retrieves multiple records using single stmt

- specifies what data is required

eg: SQL

3. Data Control Language (DCL)

- commands like GRANT, REVOKE not user access privileges

database books

S.No	Name	Ther	Mark
1	My MC		

S.No	Name	Ther	Mark
------	------	------	------

4. TCL (Transaction Control Language)

- used to manage, control transaction in a database, grouping them into logical units.
- commands like COMMIT, ROLLBACK.

↓ add → change ↶ revert

- Database Interfaces:

1. Menu based Interfaces for Web clients

Presents the user with list of options / menus that leads the user to provide a request.

2. Apps for Mobiles

Presents users with access to their data.

eg: banking, reservation.

3. Forms based Interface

Display forms to user. User can fill out the details and insert/update data

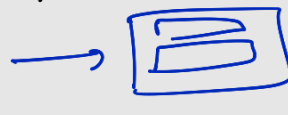


own record

DBMS will retrieve matching data for the ones filled.

4. Graphical User Interface

Displays schema in a diagrammatic form. User specifies query by manipulating diagrams.



5. Natural Lang. Interface

Has its own schema and a dictionary of imp. words.

Select id from student

6. Speech I/O

Limited use of speech as an input query and as a response

Order number → speech; your order has delivered

7. Key words based interface: uses predefined indexes to access information

eg. Google

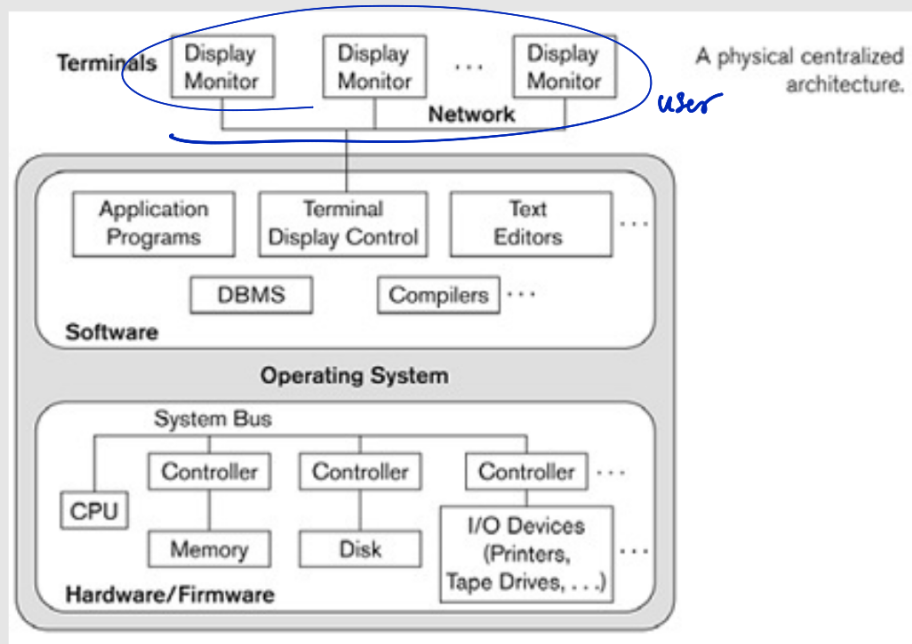
How to learn DBMS? → display.

- Centralized Client/Server Architecture for DBMSs.

Mainframe computers were replaced by users with PCs and workstations. DB system used these computers similarly to how they had used terminals.

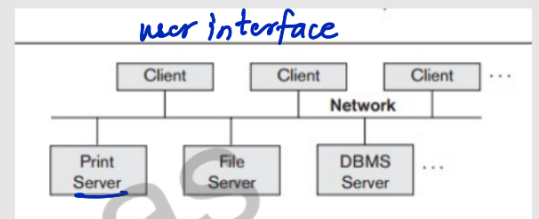
① Centralized DBMS → DBMS functionality, application program execution and user interface all in 1 machine

↓ terminals → only display user



→ Basic Client / Server Architecture

Developed to deal with computing environments where large no. of clients are connected via a network.



client
server

Specialized servers with specific functionalities

- printer server → P1, P2, P3
- web servers / email servers can be accessed by many client machines.

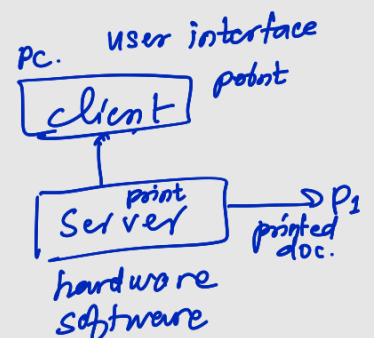
Client machines provide users with appropriate interface to utilize these servers with local processing power to run local applications

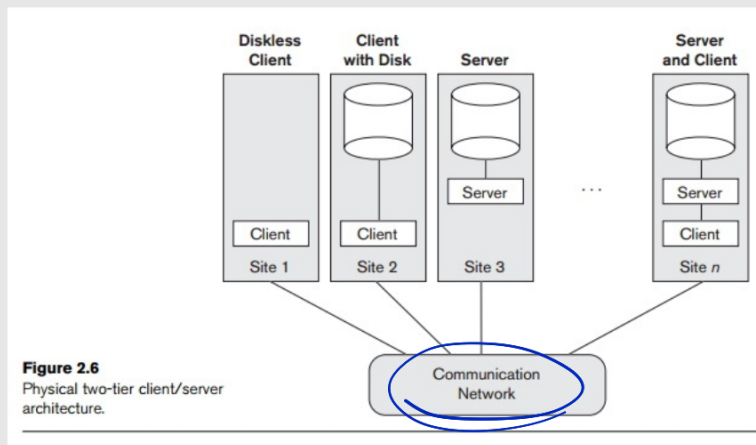
→ Physical two tier client/server architecture

Client machine provides user interface capabilities and local processing.
Server is a system containing both hardware and software that can provide services to the client machine, such as file access, printing, archival, database access.

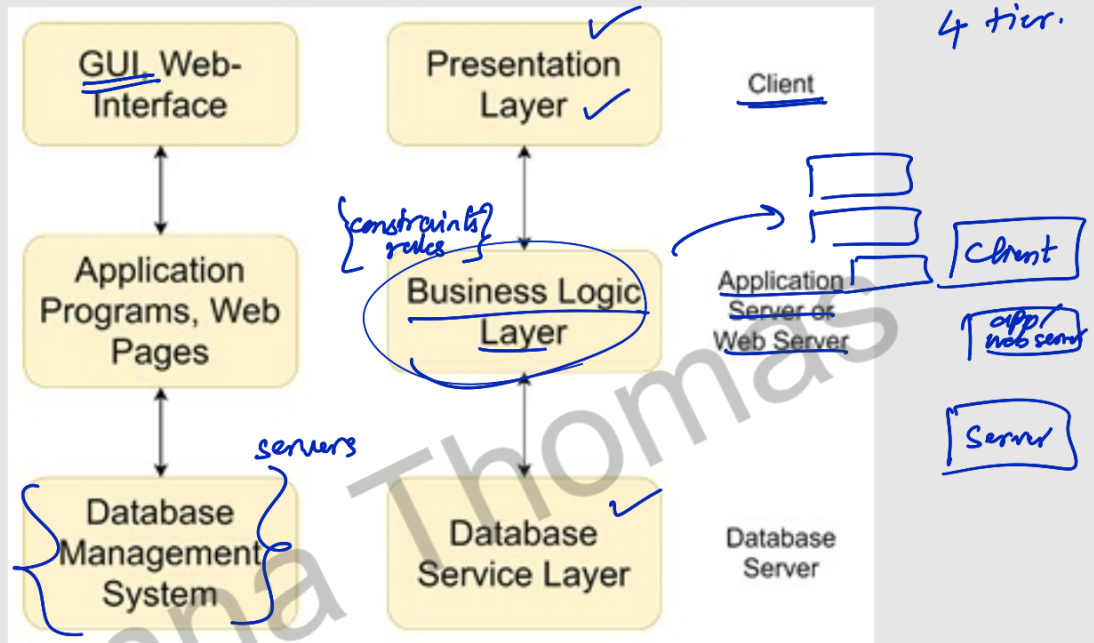
Advantages:

1. Simplicity
2. Seamless compatibility with existing systems

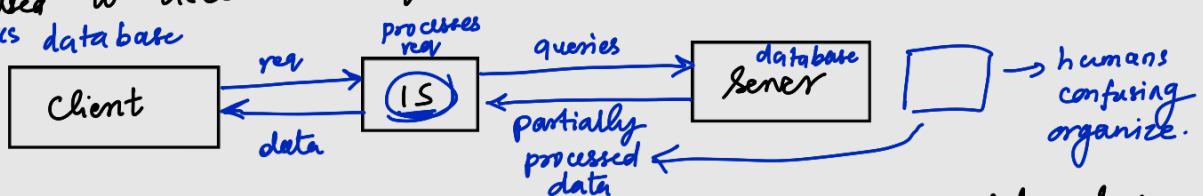




→ Three tier and n-tier architectures for Web Application



B/w client and server, there is an intermediate layer / middle tier called the application server / web server. This plays an intermediary role by running business application programs and storing business rules that are used to access data from database servers.



Presentation Layer: Displays information to user and allows data entry

Business logic layer: Handles intermediate rules and constraints before data is passed up to the user or down in DBMS

Bottom layer has data management services, middle layer acts as a web server.

→ division into multiple layers → n tier

• Conceptual Data Modelling and Database Design

- Data modelling using the Entity Relationship (ER) Model

Defines conceptual view of database. It works around real life entities and associations among them.

Allows us to move from an informal description of what users want in the database to more detailed and precise description that can be implemented in DBMS. Three basic notions are:

1. Entity

- real world object with an indpt existence, physical or conceptual
- entity type: collection of entities that have same attributes (BOOKS)

SrNo	BOOKS Name	taken
1.	Prog in C	✓
2.	OS.	✓
3.	DBMS	✓

entity set: collection of similar types of entities at a point in time (can include attributes)
it does not need to be disjoint

eg. Books related to programming, theory

RNo	Name	Ch
1	Asha	A
2	Abg	B
3	Anna	A
4	Arthy	C

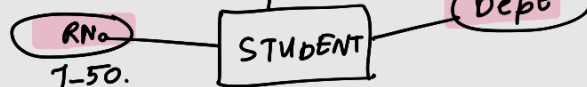
Entity set is represented by

Strong entity set
unique identify → key

Weak entity set

2. Attributes

- properties of entities
- have values / (has domain / range that can be assigned)
- each entity is described by set of attribute, data value pair



Simple

- atomic values

Composite

- made of more than 1 simple attributes

()

Derived

- values are derived from other attributes

Single valued

- contains single value

Multi-valued

- more than one value

{ }



eg: Present year -
DOB
↓
Age

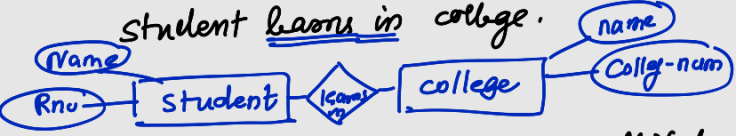
Subject
{ Prog in C, OS, DBMS }

- Key attribute: Capable of identifying each entity uniquely (Id)

Represented by linked to the entities

3. Relationships

- association among entities
- relationship set: set of relationship of similar types. Can have attributes called descriptive attributes



Represented by  in the ER diagram

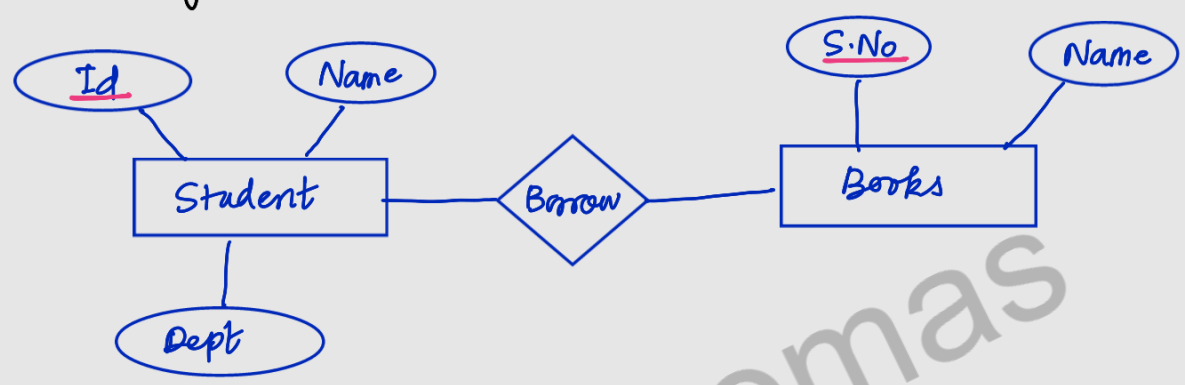
Strong relationship
both entities will be strong entity set



Weak relationship
one of the entity will be a weak entity set



ER diagram example:



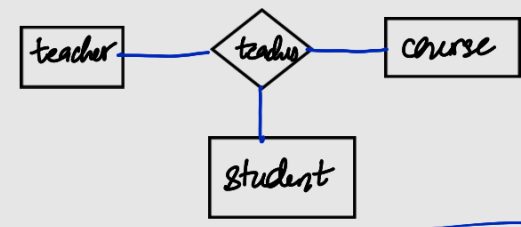
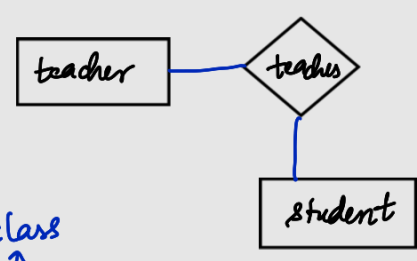
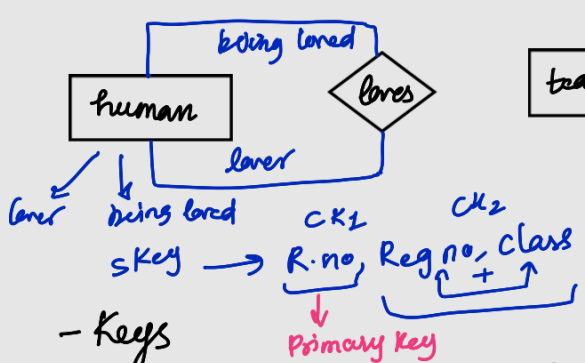
- degree: no. of participating entities in a relationship

Unary
- association with only one entity

Binary
- association among 2 entities

Ternary
- association among 3 entities

n ary.



STUDENT

R.no	Reg no.	class	no
1	202	A	Alon
2	00	A	Pravda
3	001	B	Bija

- Keys

Attribute / collection of attributes that uniquely identifies an entity



Super Key

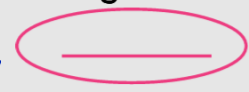
set of attributes that identifies an entity in an entity set
R.no, Reg no, class.

Candidate key

minimal super key, can have more than one candidate key.
Reg no + class. → uniquely.
R.no → uniquely.

Primary Key

one of the candidate keys chosen to identify the entity set.



- Roles

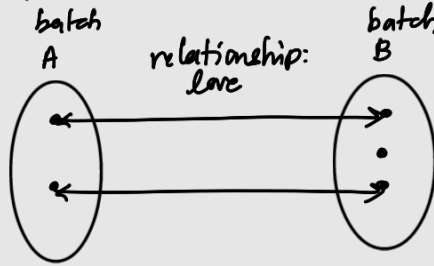
Signifies - the role that a participating entity plays in each relationship instances, clarifying meaning in recursive relationships.
(same entity participating multiple times)

- Structural Constraints

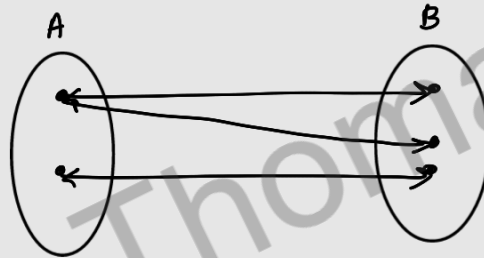
1. Cardinality

Defines - the no. of entities in an entity set that can be associated with no. of entities of other set via relationship set.

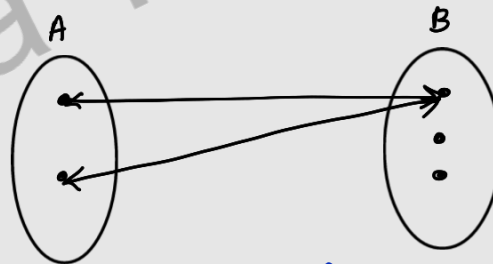
a) One to one
(1:1)



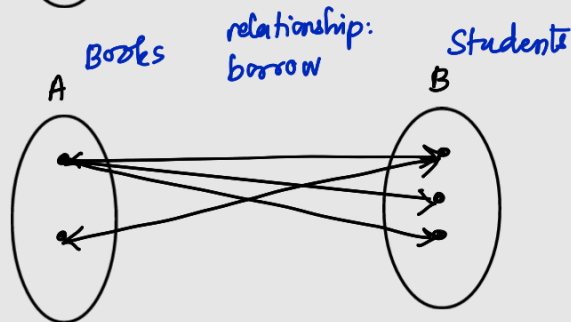
b) One to many
(1:N)



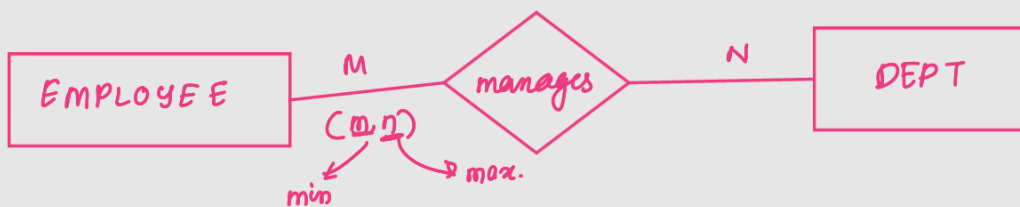
c) Many to one
(N:1)



d) Many to many
(M:N)



eg:

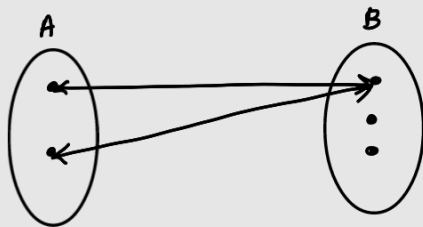


2. Participation

whether all or some entity instances are involved in the relationship.

Partial Participation

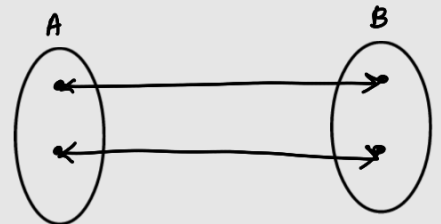
- when some of the entities in the entity set don't participate in the relation



Represented by _____

Total Participation

- when each entry in an entity set participates in a relation.



Represented by _____

Structural constraints are represented by min-max notation.

(m, n) represented above the line

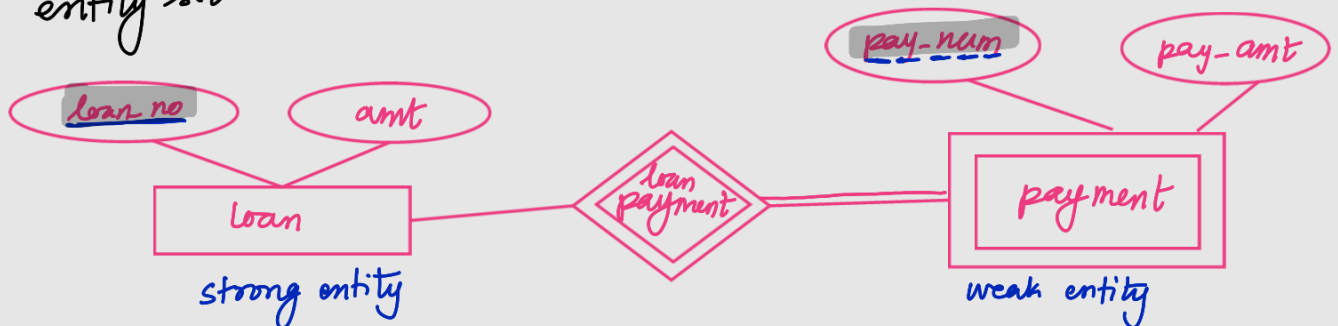
min no of times an entity appears in a relationship

max no.

- Weak Entities

Entity set may not have sufficient attribute to form a primary key. (existence dependent)

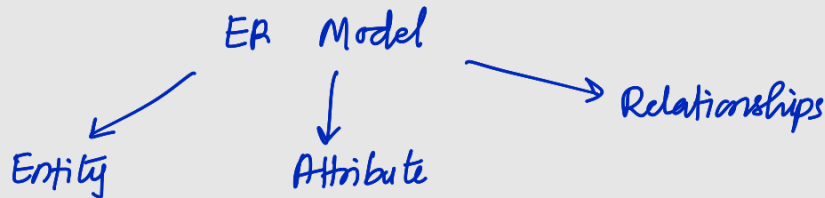
They need to be associated with another entity set called identifying or owner entity set. The discriminator is called the partial key of the entity set.



Weak entities have

- total participation
- represented by double rectangular box
- identifying relationship represented with double diamond
- partial key with dotted lines

- ER design for COMPANY database.



Problem stmt:

The COMPANY database keeps track of a company's employees, departments, and projects.

■ The Company is organized into departments. Each department has a unique name a unique number, and a particular employee who manages the department. We keep track of the start date when that employee began managing the department. A department may have several locations. → multi

■ A department controls a number of projects, each of which has a unique name a unique number, and a single location.

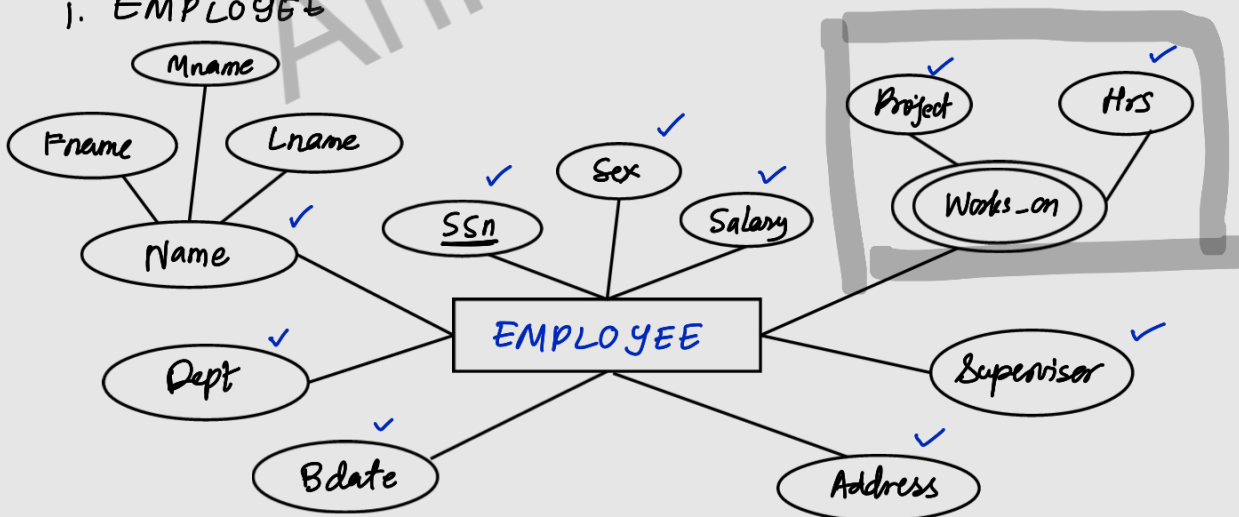
■ We store each employee's name, Social Security number, address, salary, sex (gender) and birth date. An employee is assigned to one department, but may work on several projects, which are not necessarily controlled by the same department. We keep track of the current number of hours per week that an employee works on each project. Employee has supervisor who supervises work.

■ We want to keep track of the dependents of each employee for insurance purposes. We keep each dependent's first name, sex, birth date, and relationship to the employee.

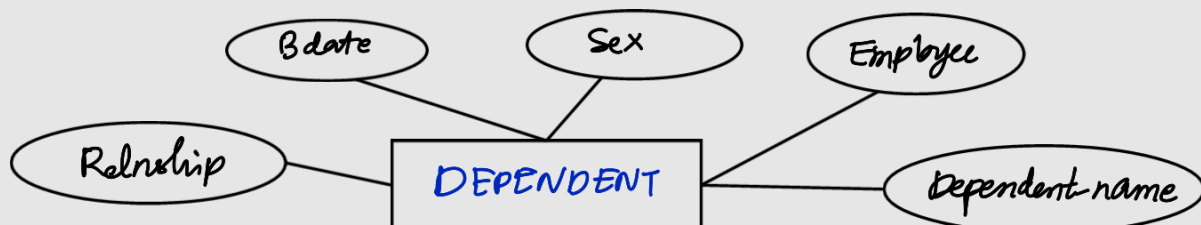
- ① Find out the entity
- ② Entity attributes
 - simple
 - composite
 - derived
- ③ relationships define
 - b/w whom
 - name
 - cardinality
 - participation

A) Entities and Attributes

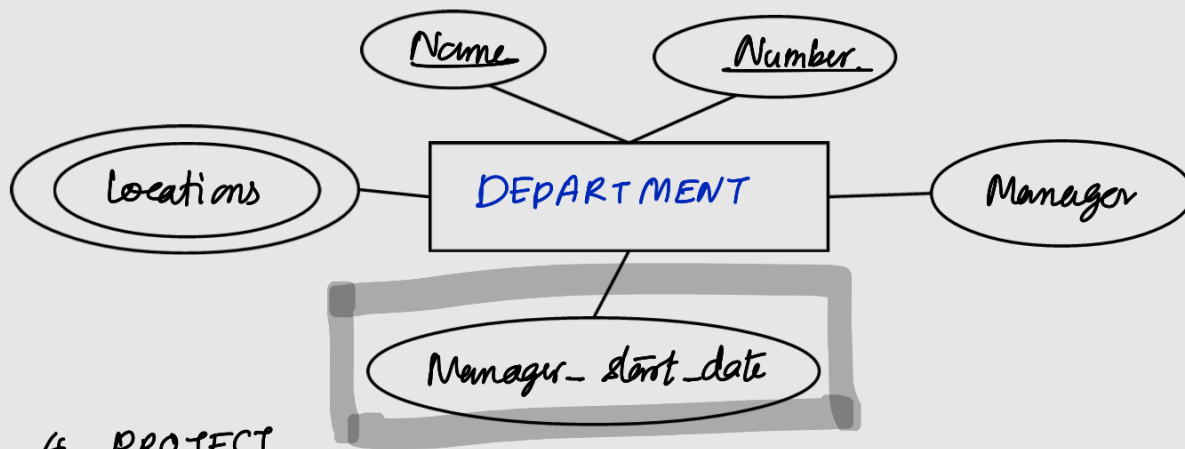
1. EMPLOYEE



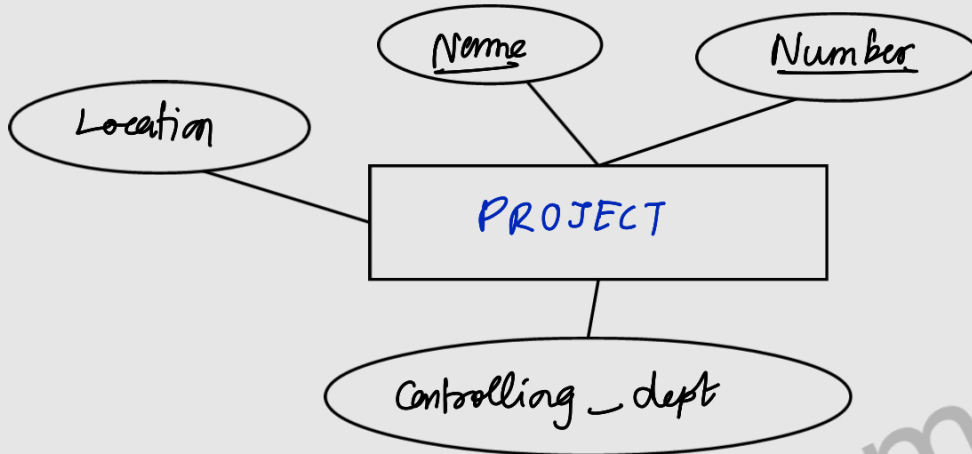
2. DEPENDENT



3. DEPARTMENT

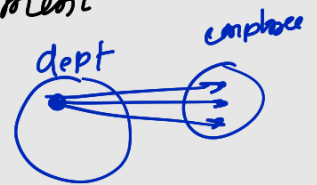


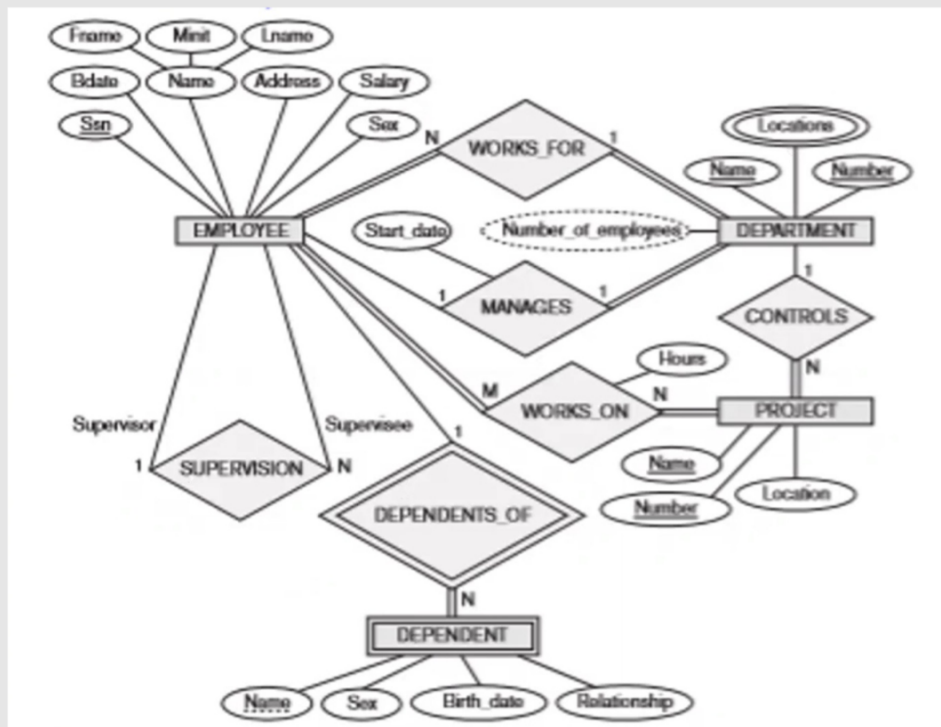
4. PROJECT



B) Relationships

- ✓ - manages, 1:1 b/w employee and department
(Employee participation is partial)
- ✓ - works-for, 1:N b/w dept and employee
(Both have total participation)
- ✓ - controls, 1:N b/w dept and project
(Project has total participation whereas dept is partial)
- ✓ - supervision, 1:N b/w employees
(both participation are partial)
- ✓ - works-on, M:N b/w project and employee (total)
- ✓ - depends-on, 1:N b/w employee & dependent
(identifying relationship for dependent, employee is partial)





- ① draw the entity with attributes
- ② relationships define
→ check for attributes additions
- ③ draw relations
→ b/w whom
→ strong/weak
→ part
→ card.

Anna Thomas